PANEL INTERVIEW · AI SOLUTIONS ENGINEER · LLNL

Production-Grade RAG & LLM Systems

Architecting for deterministic outcomes, governance & trust

Anshul Sharma

Anshul Sharma — from systems depth to GenAI architecture

Staff Software Engineer / Technical Lead · M.S. Software Engineering · 13 years in production systems

Author of *RAG in Production* · Inventor of the **ILEI Architecture** (USPTO Provisional 63/076,477)

Distributed Systems
scale, reliability, failure modes

→

Productivity Apps
large-scale user-facing products

→

Agentic Systems
tool use, orchestration, control

→

RAG / GenAI
retrieval, evaluation, guardrails

What I actually do at this level

- Set **system architecture** & define "good" in measurable terms
- Align **data owners, security, and domain experts** before we build
- Lead through **evaluation discipline**, not just code output
- Own the hard tradeoffs: latency vs. accuracy, cost vs. control

Operating principle

*"Treat the LLM as the **least reliable component** in the system — and engineer everything around containing and measuring that uncertainty."*

Everyone starts here — the naive RAG

THE DEMO PIPELINE — ONE STRAIGHT LINE

Documents → Fixed-size chunk → Embed → Vector DB → Top-k retrieve → LLM → Answer

- ✕ **Fixed chunking** severs tables, code, and claims from their caveats
- ✕ **Pure vector search** misses exact terms — part numbers, acronyms
- ✕ **No re-ranking** — right doc buried below noise
- ✕ **No access control** — retrieves chunks the user can't see
- ✕ **No evaluation** — hallucinations ship silently
- ✕ **No guardrails / citations** — confident, unsupported answers

press → to productionize

Works in a 20-minute demo. Fails the moment it meets real data, real users, and real

Retrieval is the system. The LLM is the last 10%.

Retrieval — the real lever

- **Hybrid search:** dense vectors + sparse BM25 — pure semantic quietly fails on part numbers, acronyms, exact terms
- **Re-ranking** fixes ordering — consistently beats a bigger, costlier model
- **Structure-aware chunking** — don't sever tables, code, or a claim from its caveat
- When it hallucinates, my first question is "show me the retrieved context" — not "upgrade the model"

Evaluation — the deployment gate

- **Retrieval:** context **precision** & **recall** — right evidence, low noise
- **Generation: faithfulness** — is every claim grounded in retrieved context?
- **Answer relevance** — did we address the actual question?
- Curated eval set built **with domain experts**; run on every change; **gate deploys on it like a test suite**

> *"A fluent, confident, unsupported answer is the most dangerous output a RAG system can produce. I'd rather it say 'I don't have enough information' than guess — refusal is a feature, not a failure."*

3 · MOST RECENT ENTERPRISE DATA + AI PROJECT

Unified knowledge assistant — Wiki · Slack · Jira

Engineers lost hours hunting answers fragmented across three systems — tribal knowledge locked in old threads & tickets. One grounded, access-aware assistant over all three.

Wiki / Docs

Challenge: staleness, deep nesting, mixed HTML/MD

Handling: section-aware chunking · recency decay on stale pages · canonical-URL dedup

Slack (public channels)

Challenge: conversational noise, threads, no ground truth

Handling: thread reconstruction · resolve @mentions/links · **public-only ACL filter** · Q&A-pair extraction

Jira Tickets

Challenge: structured fields + freeform comments

Handling: field-aware parsing · weight resolved/accepted answers · link-graph context

▼ normalize · ACL-tag · embed ▼

KNOWLEDGE PLANE — UNIFIED INDEX

Source connectors
incremental sync
→
Per-source chunking
+ ACL & freshness tags
→
Embed
cross-source
→
Vector + metadata store

What it changed — and what I owned

↓80%	30,000+	0.92	3 → 1
SEARCH TIME	MAN-HOURS SAVED / YEAR	FAITHFULNESS (EVAL SET)	SYSTEMS TO SEARCH

Architecture decisions I owned

- Split **knowledge vs. serving plane**; **ACL enforced at ingestion + retrieval** (public-only Slack)
- **Per-source chunking & re-ranking** over a one-size pipeline
- Built an **eval harness with SMEs** as a CI gate — nothing shipped without it
- Met engineers **in Slack**; every answer carries source deep links

Hard tradeoff

Chose **precision over coverage** — tuned to **refuse-over-guess**, so a confident wrong answer was near-zero in eval. We accepted more "not enough information" responses to protect trust in the early rollout.

Leadership

Aligned **security, data owners & SMEs** on access policy before build; the eval gate changed how the team shipped — from "looks good" to **measured**.

Opinionated, but loyal to capabilities, not vendors

Layer	What I use / evaluate	How I decide
Models	Claude (Opus/Sonnet/Haiku), open-weight (Llama, Mistral) for in-boundary/self-host	Capability per $ & latency; can it run inside the security boundary?
Orchestration	LangChain / LlamaIndex for speed; **thin custom layer** when control matters	Frameworks to prototype; own the critical path in production
Retrieval / store	pgvector, FAISS, Milvus/Qdrant; BM25/Elastic for hybrid	Scale, filtering on ACL metadata, ops burden
Evaluation	RAGAS-style metrics, custom harnesses, LLM-as-judge (calibrated)	Must gate CI; measure retrieval & generation separately
Guardrails	Structured output / schema validation, input & output filters, citation enforcement	Refuse-over-guess; every claim traceable
Ops	Tracing (OpenTelemetry / LangSmith-style), prompt & index versioning, caching	Observability and reproducibility are non-negotiable

> *"I adopt frameworks to move fast, but I own the critical path. In a deterministic-outcomes environment, I won't hand reliability to a black box I can't trace or test."*

CLOSE

Why this maps to the Lab

- **Deterministic outcomes:** evaluation as a gate; refuse-over-guess by design
- **Data governance:** ACLs, lineage, PII handling travel with the data — in-boundary by default
- **Rigorous evaluation:** faithfulness & context precision measured continuously
- **Robust architecture:** two-plane design, owned critical path, full observability
- **Multi-disciplinary:** I align security, data owners & domain experts as part of the architecture

What I bring at the lead/architect level

Not "I can build a RAG pipeline" — but **"I can make an LLM system the Lab can defend, audit, and trust at scale,"** and bring the org along to do it.

Thank you. I'd love to dig into any of these.